

Государственный Университет Молдовы
Факультет Математики и Информатики
Департамент Информатики

Индивидуальная работа

по курсу «Продвинутая WEB-разработка»
тема: «Веб-приложение "Дневник настроения"»

Выполнил студент группы IA2404:

Пармакли Леонид

Проверил преподаватель:

Борис Вишневский

Кишинёв 2026

ОГЛАВЛЕНИЕ

ЦЕЛЬ РАБОТЫ	3
ХОД РАБОТЫ	3
1. Постановка задачи	3
2. Архитектура приложения	4
3. Подключение к базе данных.....	5
4. Миграция базы данных	6
5. Репозитории и сущности.....	8
6. Обработка форм	9
7. Twig-шаблоны	10
8. Публичный и приватный контент	11
9. Система аккаунтов и сессия.....	12
10. Процесс авторизации.....	13
11. Меры безопасности.....	14
ИСПОЛЬЗОВАННЫЕ ТЕХНОЛОГИИ	15
ВЫВОД	15
СКРИНШОТЫ	Ошибка! Закладка не определена.
БИБЛИОГРАФИЯ	18

ЦЕЛЬ РАБОТЫ

Целью данной работы является разработка веб-приложения с использованием языка программирования PHP, системы управления базами данных PostgreSQL и шаблонизатора Twig. В ходе выполнения работы необходимо освоить принципы создания динамических веб-приложений, реализовать взаимодействие с реляционной базой данных, а также обеспечить безопасность обработки пользовательских данных.

Особое внимание уделяется реализации системы аутентификации пользователей, включающей регистрацию, вход в систему и управление сессиями. Также целью работы является изучение принципов разграничения прав доступа, при которых различные категории пользователей (обычные пользователи и администраторы) имеют доступ к различным функциям приложения.

Дополнительно в рамках работы требуется реализовать полный цикл операций CRUD (создание, чтение, обновление и удаление данных), а также обеспечить защиту приложения от типовых угроз, таких как SQL-инъекции, CSRF-атаки и автоматизированные запросы (боты).

В результате выполнения работы должно быть создано функциональное, безопасное и расширяемое веб-приложение с модульной архитектурой.

ХОД РАБОТЫ

1. Постановка задачи

В рамках индивидуальной работы требуется разработать веб-приложение, обеспечивающее взаимодействие пользователей с системой через браузер с использованием серверной логики на языке PHP.

Основная задача заключается в создании приложения, которое позволяет пользователям регистрироваться, входить в систему и работать с данными, хранящимися в базе данных. В качестве предметной области было выбрано приложение «Дневник настроения», позволяющее пользователям фиксировать своё эмоциональное состояние и сопутствующую информацию. Функциональные требования к приложению включают:

- реализацию механизма регистрации и аутентификации пользователей;
- хранение учетных данных в базе данных с использованием безопасного хеширования паролей;
- предоставление доступа к защищённым разделам только авторизованным пользователям;

- наличие общедоступной части приложения, отображающей данные без необходимости входа в систему;
- реализацию формы создания ресурса (записи дневника), содержащей несколько полей различных типов;
- организацию административной части приложения с расширенными правами доступа.

Приложение должно поддерживать две роли пользователей: обычный пользователь — имеет доступ только к своим данным, и администратор — имеет доступ ко всем данным и дополнительным функциям управления системой. Также в рамках задачи необходимо обеспечить выполнение требований безопасности:

- валидацию пользовательских данных;
- защиту от SQL-инъекций;
- защиту от CSRF-атак;
- использование CAPTCHA для защиты от автоматизированных запросов;
- использование сессий для управления доступом

Дополнительно требуется реализовать модульную архитектуру приложения, обеспечивающую удобство сопровождения и расширения. В качестве архитектурного подхода используется MVC-подобная модель, при которой логика обработки данных, доступ к базе данных и представление разделены на отдельные компоненты

2. Архитектура приложения

Разрабатываемое веб-приложение построено на модульной архитектуре с разделением ответственности между основными компонентами. В основе используется MVC-подобный подход, при котором логика обработки данных, доступ к базе данных и пользовательский интерфейс разделены. Основной точкой входа является файл `index.php`, расположенный в директории `public`. Все HTTP-запросы обрабатываются через данный файл, который выполняет роль маршрутизатора. Общий процесс обработки запроса:

HTTP-запрос → `index.php` → сервис → репозиторий → база данных → Twig → HTML-ответ

Проект организован следующим образом:

<code>public/</code>	— точка входа
<code>src/</code>	— бизнес-логика
<code>templates twig/</code>	— шаблоны интерфейса
<code>migrations/</code>	— миграции базы данных

Основные компоненты:

- **Репозитории** - Репозитории отвечают за взаимодействие с базой данных. В приложении реализованы: UserRepository — работа с пользователями, MoodEntryRepository — работа с записями дневника. Они инкапсулируют SQL-запросы и используют PDO для выполнения операций
- **Сервисы** - Сервисы реализуют бизнес-логику приложения. В приложении реализованы: AuthService — регистрация, вход, работа с сессиями и MoodEntryFormHandler — обработка форм записей.
- **Валидация** - Для проверки пользовательских данных используются валидаторы AuthValidator и MoodEntryValidator. Они выполняют проверку корректности данных перед их обработкой и сохранением
- **Представление (Twig)** - Для формирования интерфейса используется шаблонизатор Twig. Он позволяет отделить HTML-разметку от PHP-кода и упростить разработку пользовательского интерфейса.
- Основные шаблоны включают страницы входа, регистрации, списка записей и административной панели.
- **Управление доступом** - Для ограничения доступа используется компонент Middleware, проверяющий авторизацию пользователя и роль пользователя (user/admin). Это позволяет разделить доступ к публичным и защищённым разделам.
- **Сессии** - Для хранения данных пользователя применяется механизм сессий PHP (\$_SESSION). В сессии сохраняется информация о текущем пользователе и его роли, что позволяет реализовать авторизацию.
- **Миграции** - Для управления структурой базы данных используется механизм миграций. Он позволяет автоматически создавать и изменять таблицы, а также отслеживать выполненные изменения.

3. Подключение к базе данных

Для хранения данных в приложении используется реляционная система управления базами данных PostgreSQL. Выбор данной СУБД обусловлен её надёжностью, поддержкой стандартов SQL и широкими возможностями для работы с данными. Взаимодействие с базой данных реализовано с использованием расширения PDO (PHP Data Objects). Данный подход обеспечивает единый интерфейс для работы с различными СУБД и позволяет использовать подготовленные выражения, что повышает безопасность приложения. Подключение к базе данных инкапсулировано в отдельном классе Database, расположенном в директории

src/Core. Это позволяет централизовать управление соединением и избежать дублирования кода. Основные параметры подключения (хост, имя базы данных, пользователь и пароль) выносятся в конфигурационный файл config.php, что упрощает настройку приложения.

Пример строки подключения (DSN) и создание подключения:

```
$config = require __DIR__ . '/../../config.php';
$db = $config['db'];

$dsn = sprintf(
    'pgsql:host=%s;port=%d;dbname=%s',
    $db['host'],
    $db['port'],
    $db['dbname']
);

self::$connection = new PDO(
    $dsn,
    $db['user'],
    $db['password'],
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]
);
```

Для повышения надёжности и безопасности используются дополнительные параметры:

- включён режим выброса исключений (PDO::ERRMODE_EXCEPTION);
- отключена эмуляция подготовленных выражений;
- используется кодировка UTF-8

Пример настройки соединения:

```
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
```

Использование PDO позволяет применять подготовленные выражения, при которых SQL-запрос и данные передаются отдельно. Это предотвращает возможность SQL-инъекций и повышает безопасность обработки пользовательского ввода. Таким образом, подключение к базе данных реализовано с использованием современного и безопасного подхода, обеспечивающего надёжную работу приложения и защиту данных.

4. Миграция базы данных

Для управления структурой базы данных в приложении используется механизм миграций. Миграции позволяют автоматически создавать и изменять таблицы базы данных, а

также отслеживать применённые изменения. В проекте реализован собственный механизм миграций без использования сторонних библиотек. Основной скрипт `migrate.php` последовательно обрабатывает файлы миграций, расположенные в директории `migrations/`. Каждая миграция представляет собой отдельный PHP-файл, содержащий функцию `up()`, в которой описаны SQL-запросы для создания или изменения структуры базы данных. Пример миграции:

```
function up(PDO $pdo): void
{
    $pdo->exec("
        CREATE TABLE IF NOT EXISTS moods (
            id SERIAL PRIMARY KEY,
            code VARCHAR(32) NOT NULL UNIQUE,
            title VARCHAR(64) NOT NULL,
            icon VARCHAR(16) NOT NULL
        );
    ");
}
```

Для отслеживания применённых миграций используется специальная таблица `schema_migrations`, в которой хранится список уже выполненных файлов. Это позволяет избежать повторного выполнения одних и тех же миграций. Алгоритм работы системы миграций выглядит следующим образом:

1. Получение списка файлов из директории `migrations/`;
2. Сортировка файлов по имени;
3. Проверка, была ли миграция уже применена;
4. Выполнение миграции при необходимости;
5. Сохранение информации о выполненной миграции в базе данных.

Применение каждой миграции выполняется в рамках транзакции. В случае возникновения ошибки изменения откатываются, что обеспечивает целостность данных. В проекте реализованы следующие миграции:

- создание таблиц `moods` и `entries`;
- добавление таблицы `users`;
- добавление связи между пользователями и записями (`user_id`);
- создание таблицы `schema_migrations`.

Использование миграций позволяет автоматизировать настройку базы данных, упростить перенос проекта на другую среду, контролировать изменения структуры базы данных, обеспечить стабильность работы приложения. Таким образом, механизм миграций является важной частью архитектуры приложения и обеспечивает удобное управление структурой базы данных.

5. Репозитории и сущности

В приложении реализован слой репозитория, отвечающий за взаимодействие с базой данных. Данный слой инкапсулирует SQL-запросы и предоставляет удобный интерфейс для работы с данными, что позволяет отделить бизнес-логику от операций доступа к базе данных. Основная идея использования репозитория заключается в том, что все операции чтения и записи данных выполняются через отдельные классы, а не напрямую в коде приложения. Это повышает читаемость, упрощает сопровождение и позволяет централизованно управлять логикой работы с базой данных. В проекте реализовано два репозитория. `UserRepository` — отвечает за работу с пользователями. `MoodEntryRepository` — отвечает за работу с записями дневника настроения.

`UserRepository` - Данный класс выполняет операции, связанные с пользователями:

- создание нового пользователя;
- поиск пользователя по имени или email;
- получение пользователя по идентификатору;
- получение списка пользователей;
- удаление пользователя.

```
public function findById(int $id): ?array
{
    $stmt = $this->pdo->prepare("
        SELECT id, username, email, password_hash, role, created_at, updated_at
        FROM users
        WHERE id = :id
    ");
    $stmt->execute([':id' => $id]);
    $user = $stmt->fetch();
    return $user ?: null;
}
```

`MoodEntryRepository` - Данный класс реализует работу с записями дневника настроения:

- создание записи;
- получение записей пользователя;
- получение всех записей (для администратора);
- обновление записи;
- удаление записи;
- поиск по записям.

Также в репозитории реализована логика разграничения доступа: обычный пользователь может работать только со своими записями, тогда как администратор имеет доступ ко всем данным. Пример метода создания записи:

```
public function createRecord(array $data): int
{
    $stmt = $this->pdo->prepare("
        INSERT INTO entries (user_id, title)
        VALUES (:user_id, :title)
        RETURNING id
    ");

    $stmt->execute([
        ':user_id' => $data['user_id'],
        ':title' => $data['title']
    ]);

    return (int)$stmt->fetchColumn();
}
```

Что касается сущностей данных, в рамках проекта используются следующие основные сущности: пользователь (User), запись дневника (Entry), настроение (Mood). Хотя отдельные классы сущностей не реализованы, данные представлены в виде ассоциативных массивов, получаемых из базы данных через PDO. Такой подход упрощает реализацию и подходит для небольших приложений. Связи между сущностями:

- один пользователь может иметь множество записей;
- каждая запись связана с одним настроением;
- таблица настроений используется как справочник.

Использование паттерна «Репозиторий» позволяет эффективно изолировать бизнес-логику от работы с базой данных, централизуя SQL-запросы и избавляя проект от дублирования кода. Такой подход не только упрощает масштабирование и изменение структуры хранилища, но и делает архитектуру приложения более гибкой, закладывая фундамент для стабильной и удобной поддержки системы.

6. Обработка форм

В веб-приложении формы являются основным способом взаимодействия пользователя с системой. С их помощью реализованы такие операции, как регистрация, вход в систему, создание и редактирование записей дневника. Передача данных от клиента к серверу осуществляется с использованием метода POST, что позволяет безопасно передавать пользовательский ввод.

Обработка формы начинается с отправки данных пользователем через браузер. После этого запрос поступает в файл `index.php`, который выполняет роль центрального обработчика. На основе параметров запроса определяется необходимый сценарий обработки. Перед дальнейшей обработкой выполняется проверка CSRF-токена, что позволяет защитить приложение от поддельных запросов.

Полученные данные передаются в соответствующий валидатор, где проверяются на корректность. В процессе валидации контролируется заполненность обязательных полей, длина строк, формат данных (например, адрес электронной почты или дата), а также допустимость значений отдельных полей. В случае обнаружения ошибок формируется список сообщений, который возвращается пользователю, а форма отображается повторно с ранее введенными данными.

После успешной валидации данные передаются в сервисный слой приложения. Для работы с записями дневника используется класс `MoodEntryFormHandler`, который выполняет нормализацию данных и передает их в репозиторий для сохранения в базе данных. Аналогично, формы регистрации и входа обрабатываются через сервис `AuthService`, который выполняет проверку учетных данных и, в случае успешной аутентификации, создает пользовательскую сессию.

Пример обработки формы создания записи:

```
$result = $entryFormHandler->create($userId, $_POST);
```

7. Twig-шаблоны

Для формирования пользовательского интерфейса в приложении используется шаблонизатор Twig. Его применение позволяет отделить HTML-разметку от серверной логики на PHP, что делает код более структурированным, читаемым и удобным для сопровождения.

Twig обрабатывает данные, переданные из PHP, и формирует итоговую HTML-страницу, которая отображается в браузере пользователя. Такой подход позволяет избежать смешивания PHP-кода с HTML и упрощает разработку интерфейса.

В проекте используется система шаблонов с наследованием. Базовый шаблон `layout.twig` содержит общую структуру страницы: заголовок, навигационное меню и контейнер для контента. Все остальные страницы наследуются от него и определяют только содержимое основной части страницы. Это позволяет избежать дублирования кода и централизованно управлять внешним видом приложения.

Пример использования наследования:

```
{% extends "layout.twig" %}

{% block content %}
<h1>Главная страница</h1>
{% endblock %}
```

В шаблонах активно используются переменные, передаваемые из PHP. Например, список записей дневника передается в Twig и затем выводится с помощью циклов:

```
{% for entry in entries %}
    <tr>
        <td>{{ entry.title }}</td>
        <td>{{ entry.mood_date }}</td>
    </tr>
{% endfor %}
```

Twig также поддерживает условные конструкции, что позволяет изменять отображение интерфейса в зависимости от состояния пользователя. Например, для отображения различных элементов меню в зависимости от роли пользователя:

```
{% if current_user %}
    <a href="index.php?page=list">Мои записи</a>
{% else %}
    <a href="index.php?page=login">Войти</a>
{% endif %}
```

Дополнительно используются фильтры Twig для форматирования данных. В проекте, например, реализован фильтр для отображения уровня энергии в удобочитаемом виде. Использование Twig обеспечивает безопасный вывод данных, так как значения автоматически экранируются, что предотвращает выполнение вредоносного кода в браузере пользователя. Таким образом, применение шаблонизатора Twig позволяет эффективно разделить логику и представление, повысить читаемость кода и упростить разработку пользовательского интерфейса.

8. Публичный и приватный контент

В приложении реализовано разделение контента на публичный и приватный, что позволяет обеспечить доступ к различным разделам системы в зависимости от статуса пользователя. Такой подход является важным элементом архитектуры веб-приложения и соответствует требованиям безопасности.

Публичная часть приложения доступна всем пользователям без необходимости авторизации. К ней относится главная страница (home), на которой отображается информация о приложении, а также динамически сформированные данные из базы данных, такие как

последние записи и статистика. Это позволяет пользователю получить общее представление о функциональности системы без регистрации.

Приватная часть приложения доступна только авторизованным пользователям. После успешного входа пользователь получает доступ к своим записям дневника, может создавать новые записи, редактировать существующие и выполнять поиск по своим данным. Доступ к этим страницам ограничен с помощью проверки сессии, что предотвращает несанкционированный доступ.

Дополнительно реализована административная часть приложения, доступная только пользователям с ролью администратора. Администратор имеет расширенные права, включая доступ ко всем записям, управление пользователями и создание новых администраторов. Это позволяет централизованно управлять системой и контролировать данные.

Разграничение доступа реализовано с использованием компонента Middleware, который проверяет наличие авторизации и роль пользователя перед выполнением действий. Например, доступ к страницам работы с записями требует авторизации, а доступ к административным функциям — наличия роли администратора.

9. Система аккаунтов и сессия

В приложении реализована система пользовательских аккаунтов, обеспечивающая регистрацию, хранение учетных данных и управление доступом к функционалу системы. Каждый пользователь имеет уникальный аккаунт, который используется для идентификации и разграничения прав доступа.

Данные пользователей хранятся в таблице `users` базы данных. Для каждого пользователя сохраняются имя, адрес электронной почты, хеш пароля и роль. Пароли не хранятся в открытом виде, а преобразуются с помощью функции `password_hash`, что обеспечивает защиту учетных данных даже в случае компрометации базы данных.

После успешной регистрации пользователь может выполнить вход в систему, указав свои учетные данные. Проверка пароля осуществляется с использованием функции `password_verify`, которая сравнивает введенный пароль с сохранённым хешем.

Для хранения информации о текущем пользователе используется механизм сессий PHP. После успешной аутентификации в глобальный массив `$_SESSION` записываются основные данные пользователя, такие как его идентификатор, имя и роль. Это позволяет системе распознавать пользователя при последующих запросах без необходимости повторного ввода данных.

Сессии позволяют сохранять состояние между отдельными HTTP-запросами, что является важным аспектом работы веб-приложений. Благодаря этому пользователь остается авторизованным до момента выхода из системы или завершения сессии.

Выход из системы реализуется путем удаления данных пользователя из сессии. После этого доступ к защищённым разделам приложения становится невозможным до повторной авторизации.

10. Процесс авторизации

Процесс авторизации в приложении представляет собой последовательность действий, направленных на проверку подлинности пользователя и предоставление ему доступа к защищённым разделам системы.

Авторизация начинается с ввода пользователем своих учетных данных в форме входа, включающей имя пользователя и пароль. Дополнительно используется CAPTCHA, которая позволяет убедиться, что запрос выполняется человеком, а не автоматизированной программой. После отправки формы данные передаются на сервер методом POST и обрабатываются в файле `index.php`. На данном этапе выполняется проверка корректности данных, а также проверка CSRF-токена, предотвращающая поддельные запросы. Далее данные передаются в сервис `AuthService`, который отвечает за обработку авторизации. Сервис обращается к `UserRepository` для поиска пользователя по имени. Если пользователь найден, выполняется проверка пароля с использованием функции `password_verify`, которая сравнивает введенный пароль с сохранённым хешем.

Пример проверки:

```
if (password_verify($password, $user['password_hash'])) {  
    $_SESSION['user'] = $user;  
}
```

В случае успешной проверки создаётся сессия, в которой сохраняются данные пользователя. После этого пользователь считается авторизованным и получает доступ к защищённым страницам приложения. Если данные введены неверно, пользователю выводится сообщение об ошибке, а форма входа отображается повторно. Доступ к защищённым разделам контролируется с помощью компонента `Middleware`, который проверяет наличие активной сессии. Если пользователь не авторизован, он перенаправляется на страницу входа. Дополнительно проверяется роль пользователя для ограничения доступа к административным функциям.

11. Меры безопасности

При разработке веб-приложения особое внимание было уделено обеспечению безопасности обработки пользовательских данных и защите системы от типовых угроз. В приложении реализован комплекс мер, направленных на предотвращение несанкционированного доступа и защиту от распространённых атак.

Одним из ключевых аспектов является безопасное хранение паролей. Вместо хранения паролей в открытом виде используется функция `password_hash`, которая преобразует пароль в хеш с применением криптографических алгоритмов. При авторизации используется функция `password_verify`, позволяющая безопасно сравнить введенный пароль с сохранённым значением.

Для защиты от SQL-инъекций все запросы к базе данных выполняются с использованием подготовленных выражений PDO. В этом случае SQL-запрос и пользовательские данные передаются отдельно, что исключает возможность внедрения вредоносного кода в запрос.

Дополнительно реализована защита от CSRF-атак. Для этого в формы добавляется уникальный токен, который сохраняется в сессии и проверяется при отправке формы. Если токен отсутствует или не совпадает, запрос отклоняется.

В приложении также используется CAPTCHA, реализованная в виде простого арифметического выражения. Пользователь должен ввести правильный ответ перед выполнением операций входа или регистрации. Это позволяет снизить риск автоматизированных атак и регистрации ботов.

Все пользовательские данные проходят обязательную валидацию на стороне сервера. Проверяется корректность формата, длина строк, наличие обязательных полей и допустимость значений. Это предотвращает попадание некорректных или вредоносных данных в систему.

Разграничение доступа реализовано с использованием ролей пользователей. Обычные пользователи имеют доступ только к своим данным, в то время как администратор обладает расширенными правами. Контроль доступа осуществляется с помощью проверки сессии и роли пользователя перед выполнением операций.

Для хранения информации о пользователе используется механизм сессий PHP. Это позволяет сохранять состояние между запросами и предотвращает необходимость повторной аутентификации. При выходе из системы данные сессии очищаются.

Также используется автоматическое экранирование данных при выводе в шаблонах Twig, что защищает приложение от XSS-атак.

ИСПОЛЬЗОВАННЫЕ ТЕХНОЛОГИИ

В ходе разработки веб-приложения были использованы следующие технологии:

- PHP — серверный язык программирования для реализации логики приложения;
- PostgreSQL — реляционная система управления базами данных;
- PDO (PHP Data Objects) — интерфейс для безопасной работы с базой данных;
- Twig — шаблонизатор для разделения логики и представления;
- HTML5 — язык разметки для создания пользовательского интерфейса;
- CSS3 — стилизация пользовательского интерфейса;
- HTTP/HTTPS — протоколы передачи данных между клиентом и сервером;
- Sessions (сессии PHP) — механизм хранения состояния пользователя;
- CSRF-токены — защита от поддельных запросов;
- CAPTCHA — защита от автоматизированных действий (ботов);
- Prepared Statements — защита от SQL-инъекций;
- Git — система контроля версий;
- Composer — менеджер зависимостей для PHP.

ВЫВОД

В ходе выполнения индивидуальной работы было разработано веб-приложение «Дневник настроения», реализующее основные принципы построения современных веб-систем. В процессе работы были освоены методы взаимодействия с реляционной базой данных PostgreSQL с использованием PDO, реализованы механизмы регистрации и авторизации пользователей, а также разграничение прав доступа на основе ролей. Особое внимание было уделено архитектуре приложения, основанной на разделении логики, представления и доступа к данным, что позволило сделать систему более структурированной и расширяемой. В приложении реализованы основные операции CRUD, обработка пользовательских форм с валидацией, поиск данных и динамическое формирование интерфейса с использованием шаблонизатора Twig. Также были применены ключевые меры безопасности, включая хеширование паролей, защиту от SQL-инъекций, CSRF-атак и использование CAPTCHA. В результате создано функциональное, безопасное и удобное в использовании веб-приложение, соответствующее поставленным требованиям и демонстрирующее практическое применение технологий веб-разработки.

ПРИЛОЖЕНИЕ А

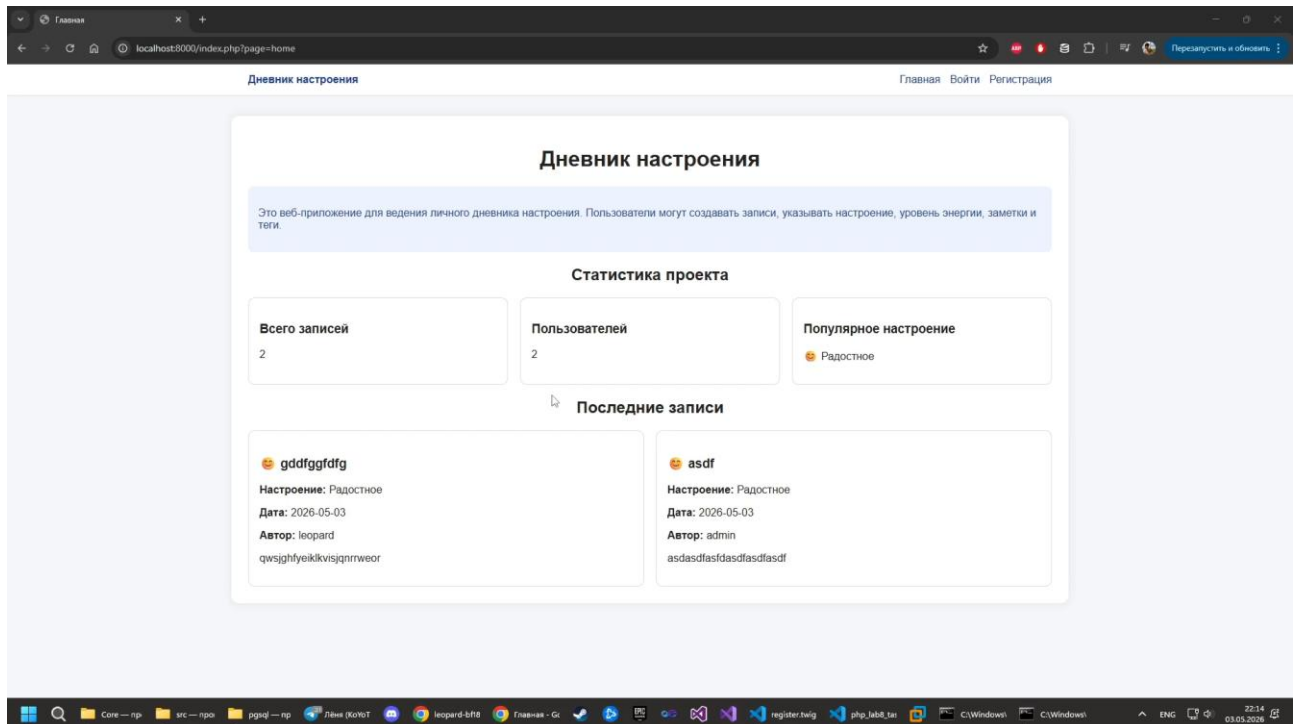


Рисунок 1- главная страница без входа

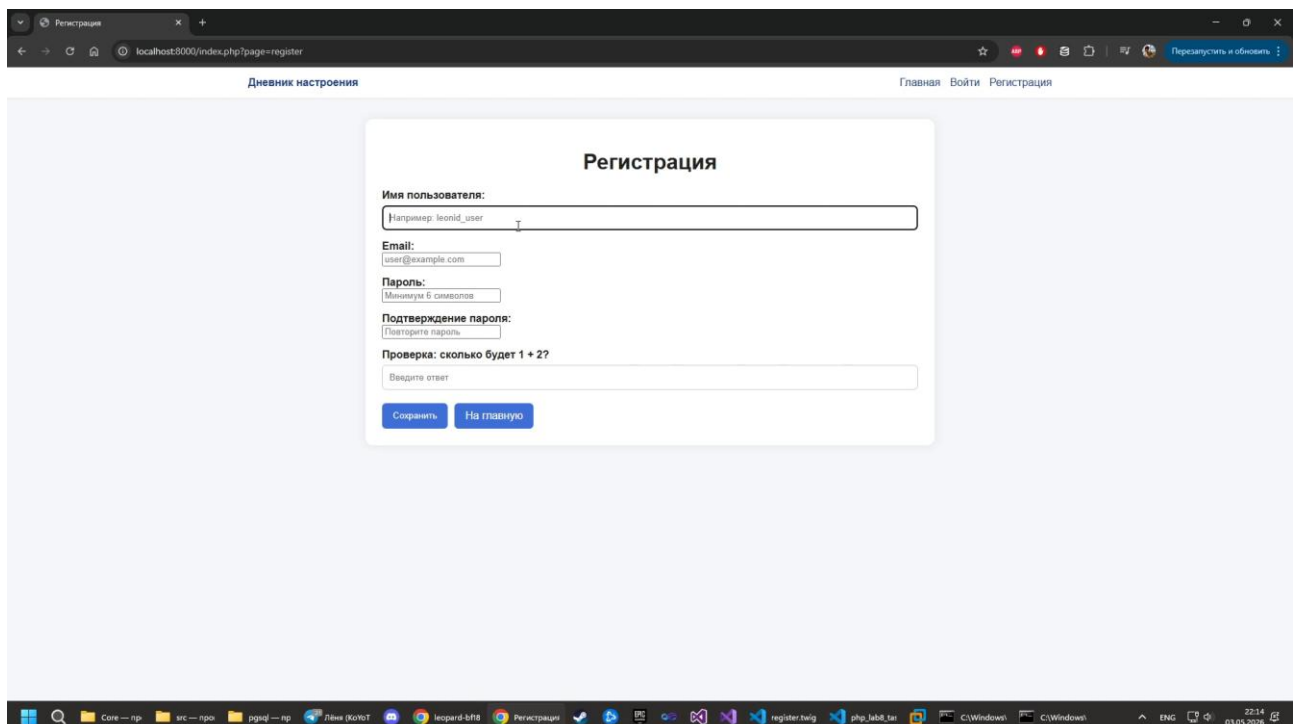


Рисунок 2 - страница регистрации аккаунта

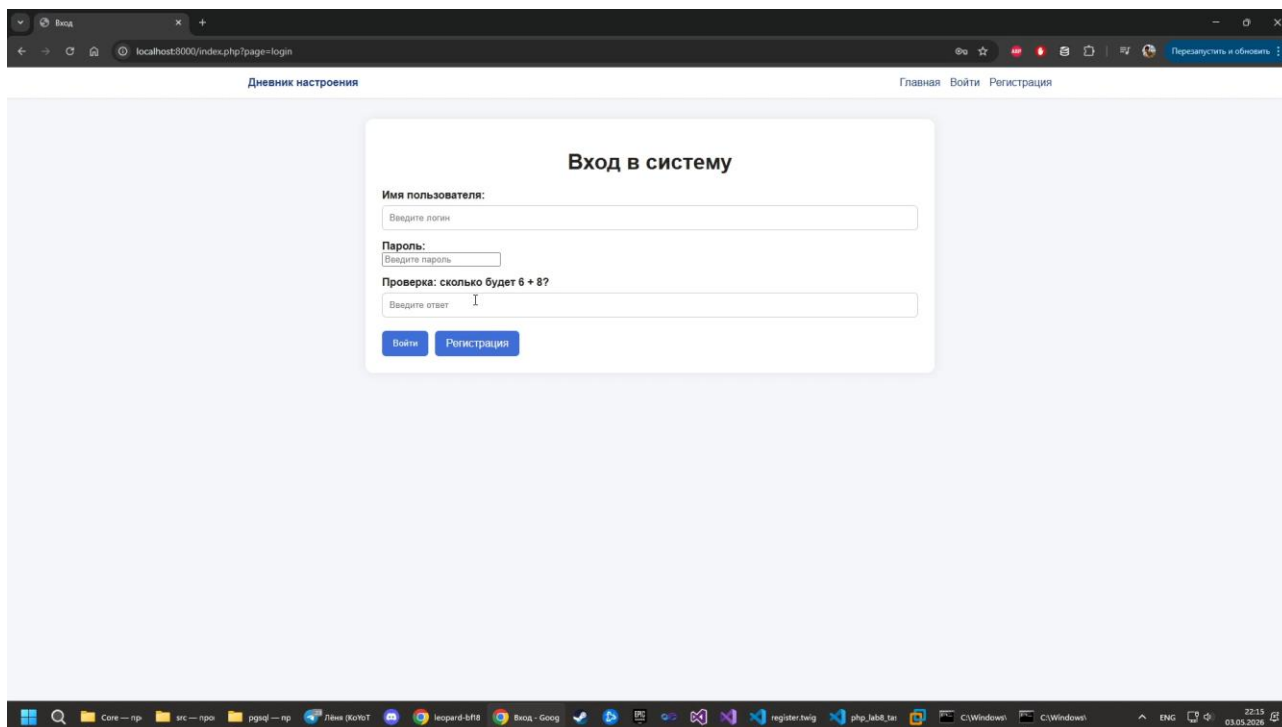


Рисунок 3 - страница входа в аккаунт

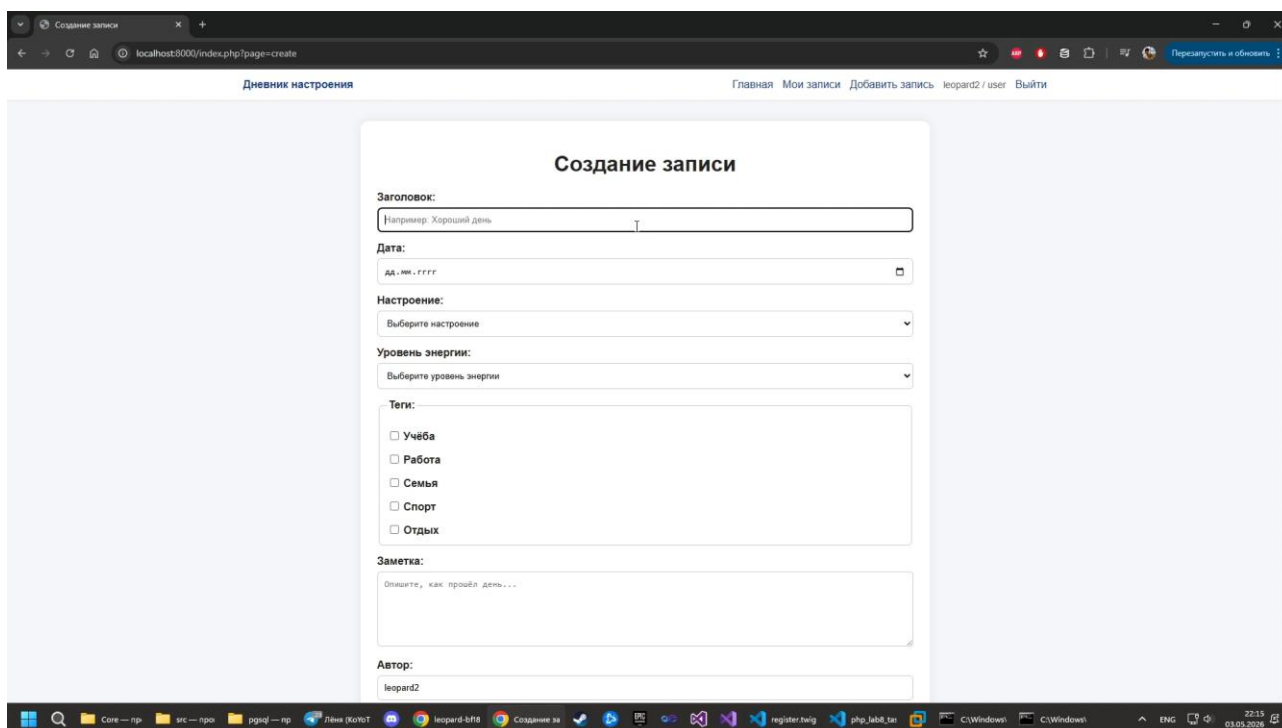


Рисунок 4 - страница создания записи

БИБЛИОГРАФИЯ

Изучение PHP

- <https://www.php.net/>
- <https://twig.symfony.com/>
- <https://www.postgresql.org/docs/>
- <https://developer.mozilla.org/>

Изучение HTML/CSS

- <https://metanit.com/web/html5/>
- https://www.w3schools.com/css/css_grid_container.asp
- <https://www.geeksforgeeks.org/html-colgroup-width-attribute/>
- https://developer.mozilla.org/ru/docs/Learn/Getting_started_with_the_web/HTML_basics
- <https://htmlbook.ru/html>